

1 Solve the Tic-Tac-Toe problem using the Depth First Search technique.

Tic Tac Toe board

```
board = ["X","X","X",  
         "O","O"," ",  
         " "," "," "]
```

Winning combinations

```
wins = [  
    (0,1,2),(3,4,5),(6,7,8),  
    (0,3,6),(1,4,7),(2,5,8),  
    (0,4,8),(2,4,6)  
]
```

def check_winner(board):

for a,b,c in wins:

```
        if board[a] == board[b] == board[c] and board[a] != " ":  
            return board[a]
```

if " " not in board:

```
        return "Draw"
```

return None

result = check_winner(board)

if result == "X":

```
    print("X Wins")
```

elif result == "O":

```
    print("O Wins")
```

elif result == "Draw":

```
    print("Match Draw")
```

else:

```
    print("Game Still Running")
```

2.Show that the 8-puzzle states are divided into two disjoint sets, such that any state is reachable from any other state in the same set, while no state is reachable from any state in the other set.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

# Function to count inversions
def count_inversions(state):
    arr = [x for x in state if x != 0] # ignore blank
    inv = 0
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] > arr[j]:
                inv += 1
    return inv

# Label function (0 = even, 1 = odd)
def get_label(state):
    return count_inversions(state) % 2

# Training data
states = [
    [1,2,3,4,5,6,7,8,0],
    [1,2,3,4,5,6,8,7,0],
    [1,2,3,4,5,6,7,0,8],
    [1,2,3,5,4,6,7,8,0],
    [1,3,2,4,5,6,7,8,0],
    [1,2,3,4,6,5,7,8,0]
]

labels = [get_label(s) for s in states]

# Train ML model
model = DecisionTreeClassifier()
model.fit(states, labels)

# ---- USER INPUT ----
print("Enter 8 numbers for the 8-puzzle (1-8). Blank will be added automatically as 0.")
```

```

try:
    user_input = list(map(int, input("Enter 8 numbers separated by
space: ").split()))

    if len(user_input) != 8:
        print("Error: Enter exactly 8 numbers.")
    else:
        user_input.append(0) # add blank
        total_inv = count_inversions(user_input)
        parity = "Even" if total_inv % 2 == 0 else "Odd"

        test_state = np.array(user_input).reshape(1, -1)
        prediction = model.predict(test_state)[0]

        print("\nPuzzle State:", user_input)
        print("Total Inversions:", total_inv)
        print("Inversion Parity:", parity)

        if prediction == 0:
            print("This state belongs to EVEN set → Reachable from goal
state")
        else:
            print("This state belongs to ODD set → Not reachable from goal
state")

except ValueError:
    print("Error: Please enter only integers.")

```

3.To represent and evaluate different scenarios using predicate logic and knowledge rules

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# ---- 1. Predicate Logic: Eligibility Rules ----
# A student is eligible if:
# Attendance >= 75
# AND InternalMarks >= 40

def check_eligibility(attendance, internal_marks):
    return attendance >= 75 and internal_marks >= 40

# ---- 2. Evaluate some example students using logic ----
students = [
    {"Name": "Alice", "Attendance": 80, "InternalMarks": 45},
    {"Name": "Bob", "Attendance": 70, "InternalMarks": 50},
    {"Name": "Charlie", "Attendance": 90, "InternalMarks": 38},
    {"Name": "David", "Attendance": 78, "InternalMarks": 42},
]

print("=== Predicate Logic Evaluation ===")
for s in students:
    eligible = check_eligibility(s["Attendance"], s["InternalMarks"])
    status = "Eligible" if eligible else "Not Eligible"
    print(f'{s["Name"]}: {status}')

# ---- 3. Machine Learning: Predict Eligibility ----
# Create dataset for ML
data = [
    {"Attendance": 80, "InternalMarks": 45, "Eligible": 1},
    {"Attendance": 70, "InternalMarks": 50, "Eligible": 0},
    {"Attendance": 90, "InternalMarks": 38, "Eligible": 0},
    {"Attendance": 78, "InternalMarks": 42, "Eligible": 1},
    {"Attendance": 60, "InternalMarks": 30, "Eligible": 0},
    {"Attendance": 85, "InternalMarks": 50, "Eligible": 1},
]
```

```

df = pd.DataFrame(data)
X = df[["Attendance", "InternalMarks"]]
y = df["Eligible"]

# Train Decision Tree Classifier
model = DecisionTreeClassifier()
model.fit(X, y)

# ---- 4. Predict New Student Eligibility using ML ----
print("\n=== Machine Learning Prediction ===")
# User input
try:
    attendance = int(input("Enter student attendance (%): "))
    internal_marks = int(input("Enter student internal marks: "))

    new_student = pd.DataFrame([{"Attendance": attendance,
                                "InternalMarks": internal_marks}])
    prediction = model.predict(new_student)[0]

    status = "Eligible" if prediction == 1 else "Not Eligible"
    print(f"ML Prediction: The student is {status}")

except ValueError:
    print("Error: Please enter valid integers for attendance and marks.")

```

4. To apply the Find-S and Candidate Elimination algorithms to a concept learning task and compare their inductive biases and outputs.

Dataset: [Attendance, InternalMarks] -> Eligible

```

data = [
    ["High", "Good", "Yes"],
    ["High", "Poor", "No"],
    ["Low", "Good", "No"],
    ["High", "Good", "Yes"],
]

```

```
attributes = ["Attendance", "InternalMarks"]
```

```
# ----- FIND-S -----
```

```
def find_s(data):
```

```
    S = ["0"] * (len(data[0]) - 1) # most specific
```

```
    for example in data:
```

```
        if example[-1] == "Yes": # positive example
```

```
            for i in range(len(S)):
```

```
                if S[i] == "0":
```

```
                    S[i] = example[i]
```

```
                elif S[i] != example[i]:
```

```
                    S[i] = "?"
```

```
    return S
```

```
S_hypothesis = find_s(data)
```

```
print("Find-S Hypothesis:", S_hypothesis)
```

```
# ----- CANDIDATE ELIMINATION -----
```

```
def candidate_elimination(data):
```

```
    S = ["0"] * (len(data[0]) - 1) # specific boundary
```

```
    G = [{"?" for _ in range(len(data[0]) - 1)}] # general boundary
```

```
    # Possible values for each attribute
```

```
    attr_values = [
```

```
        ["High", "Low"], # Attendance
```

```
        ["Good", "Poor"] # InternalMarks
```

```
    ]
```

```
print("\n--- Candidate Elimination Step-by-Step ---")
```

```
for idx, example in enumerate(data):
```

```

print(f"\nProcessing example {idx+1}: {example}")

if example[-1] == "Yes": # positive example

    # Generalize S

    for i in range(len(S)):

        if S[i] == "0":

            S[i] = example[i]

        elif S[i] != example[i]:

            S[i] = "?"

    # Remove inconsistent G

    G = [g for g in G if all(g[i] == "?" or g[i] == example[i] for i in range(len(g)))]

else: # negative example

    new_G = []

    for g in G:

        for i in range(len(g)):

            if g[i] == "?":

                for val in attr_values[i]:

                    if val != example[i]:

                        new_hyp = g.copy()

                        new_hyp[i] = val

                        # Must be at least as general as S

                        consistent = all(S[j] == "?" or new_hyp[j] == S[j] for j in range(len(S)))

                        if consistent:

                            new_G.append(new_hyp)

                    elif g[i] != example[i]:

                        new_G.append(g)

    # Remove duplicates

    G = [list(x) for x in set(tuple(h) for h in new_G)]

print(f"S = {S}")

print(f"G = {G}")

return S, G

```

```
S_CE, G_CE = candidate_elimination(data)
print("\nCandidate Elimination Final Specific boundary S:", S_CE)
print("Candidate Elimination Final General boundary G:")
for g in G_CE:
    print(g)
```